

1. Объясните принцип «разделяй и властвуй» на примере алгоритма сортировки слиянием.

Сложную задачу разбивают на *меньшие, аналогичные подзадачи*, которые решаются *независимо* (обычно рекурсивно). Затем решения подзадач *объединяются* для получения решения исходной задачи.

Разделяй (Divide). Исходный массив рекурсивно разделяется на две примерно равные части. Процесс продолжается до тех пор, пока не будут получены подмассивы, состоящие из одного элемента. Массив из одного элемента считается тривиально отсортированным.

Покоряй (Conquer). Каждый полученный минимальный подмассив (из одного элемента) "решается" тривиально – он уже отсортирован.

Соединяй (Combine). Это ключевой этап сортировки слиянием – слияние (merge). Пары отсортированных подмассивов сливаются в новые отсортированные подмассивы. Процесс слияния повторяется рекурсивно вверх по дереву деления, пока не будет слит весь исходный массив в один отсортированный массив.

Благодаря принципу «разделяй и властвуй» на больших объёмах данных можно реализовать *параллельные вычисления, работу с внешней памятью*.

Страница 313 учебника.

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(right_half)

        i = j = k = 0

        # Слияние отсортированных половин
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
```

```

# Добавление оставшихся элементов из левой половины
while i < len(left_half):
    arr[k] = left_half[i]
    i += 1
    k += 1

# Добавление оставшихся элементов из правой половины
while j < len(right_half):
    arr[k] = right_half[j]
    j += 1
    k += 1

```

2. Как работает метод динамического программирования? Проиллюстрируйте на задаче о наибольшей возрастающей подпоследовательности (LIS).

Это метод решения сложных задач путём разбиения на перекрывающиеся подзадачи. Ключевая особенность — решения подзадач *запоминаются* и многократно используются, что позволяет избежать повторных вычислений.

Определение подзадачи: $dp[i]$ — длина наибольшей возрастающей подпоследовательности, оканчивающейся на элементе $a[i]$.

Рекуррентное соотношение:

$$dp[i] = 1 + \min_{j=1..n} dp[j] \quad \text{для всех } j < i, \text{ таких что } a[j] < a[i]$$

Для нахождения самой подпоследовательности: находим индекс k с максимальным $dp[k]$. Двигаемся назад, отбирая элементы, где $dp[i] = dp[k] - 1, a[i] < a[k]$ и т.д.

Свойство ДП	Реализация в задаче LIS
Перекрывающиеся подзадачи	$dp[j]$ используется многократно
Оптимальная подструктура	Решение для $dp[i]$ строится на основе оптимальных решений $dp[j]$ ($j < i$)
Запоминание решений	Таблица dp хранит решения всех подзадач
Восходящий подход	Вычисляем подзадачи от меньших индексов к большим ($i = 0 \rightarrow n - 1$)
Отсутствие повторных расчётов	Каждое $dp[i]$ вычисляется один раз и хранится в таблице

```

def findLIS(a: list[int]) -> list[int]:
    n = len(a)
    prev = [0] * n
    d = [0] * n

    for i in range(n):
        d[i] = 1
        prev[i] = -1
        for j in range(i):
            if (a[j] < a[i] and d[j] + 1 > d[i]):
                d[i] = d[j] + 1
                prev[i] = j

    pos = 0
    length = d[0]
    for i in range(n):
        if d[i] > length:
            pos = i
            length = d[i]

    answer = []
    while pos != -1:
        answer.append(a[pos])
        pos = prev[pos]
    answer.reverse()

    return answer

```

[Описание алгоритма на neerc.ifmo](https://neerc.ifmo.ru/wiki/index.php?title=Longest_Increasing_Subsequence)

3. В чём разница между жадными алгоритмами и динамическим программированием? Приведите примеры.

Критерий	Жадные алгоритмы	Динамическое программирование (ДП)
Основная стратегия	Локально оптимальный выбор на каждом шаге	Комбинирование решений перекрывающихся подзадач
Оптимальность	Требует доказательства (не всегда оптимально)	Гарантирует оптимальное решение

Критерий	Жадные алгоритмы	Динамическое программирование (ДП)
Сложность	Обычно низкая ($O(n \log n)$, $O(n)$)	Часто полиномиальная ($O(n^2)$, $O(n^3)$)
Память	Минимальная ($O(1)$ - $O(n)$)	Значительная (таблицы решений)
Обратная связь	Решения не пересматриваются	Учитывает все варианты через подзадачи
Структура задачи	Свойство оптимальной подструктуры + жадный выбор	Оптимальная подструктура + перекрывающиеся подзадачи

Задача о рюкзаке (ДП)

Набрать предметы с максимальной стоимостью без превышения веса W .

Пусть (стоимость, вес) = [(1500, 1), (2000, 3), (4000, 4)], $W=4$.

Жадный подход. Брать предметы с максимальным соотношением стоимость/вес = [1000, 666.67, 1500]. Выберется: [(1500, 1), (2000, 3)].

ДП. $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$

Вместимость>	1	2	3	4
1 предмет	1500	1500	1500	1500
2 предмета	1500	1500	2000	3500
3 предмета	1500	1500	2000	4000

Глава 8 и 9 книги «Грокаем алгоритмы».

Задача о выборе интервалов (жадный)

Выбрать максимальное количество непересекающихся интервалов.

Жадный подход. Сортируем по времени окончания, выбираем как можно раньше заканчивающиеся.

ДП. Подзадачи независимы (выбор одного интервала не влияет на структуру оставшихся).

4. Как работает хеширование с цепочками? Оцените среднее время поиска при равномерном распределении ключей.

Хеш-таблица представляет собой **массив размера B** , где каждая ячейка содержит указатель на **связный список** (цепочку).



По ключу получаем индекс: $index = h(key) \% B$. Пусть h работает за $O(1)$.

Вставка $O(1)$ – элемент добавляется в начало связного списка в ячейке $table[index]$.

Поиск: в связном списке $table[index]$ выполняется линейный поиск по ключу.

Если сегменты примерно одинаковы по размеру, то в этом случае списки всех сегментов должны быть наиболее короткими при данном числе сегментов. Если исходное множество состоит из N элементов, тогда средняя длина списков будет N / B элементов. Если можно оценить величину N и выбрать B как можно ближе к этой величине, то в каждом списке будет один-два элемента. Тогда время выполнения операторов словарей будет малой постоянной величиной, зависящей от N (или, что эквивалентно, от B).

[Страница 117 учебника.](#)

5. Докажите корректность алгоритма Дейкстры для нахождения кратчайших путей.

Пусть есть ориентированный граф $G(V, E)$, у которого все дуги имеют неотрицательные метки (стоимости дуг), а одна вершина определена как источник *source*. Задача состоит в нахождении стоимости кратчайших путей D от источника ко всем другим вершинам графа G (здесь длина пути определяется как сумма стоимостей дуг, составляющих путь).

Алгоритм. Изначально $D[source] = 0$, остальные $D[v] = \infty$. На каждом шаге выбираем вершину $current$, которой нет в S с минимальным до неё расстоянием. Для каждого соседа $neighbor$ вершины посчитать путь как

$$\min(D[neighbor], D[current] + w(current, neighbor))$$

В конце шага добавить $current$ в S .

Инвариант.

На каждом шаге для множества посещенных вершин S найден кратчайший путь через вершины множества S .

По индукции. При пустом S : $D[s] = 0$. Пусть, верно, при k вершинах. Добавим вершину u .

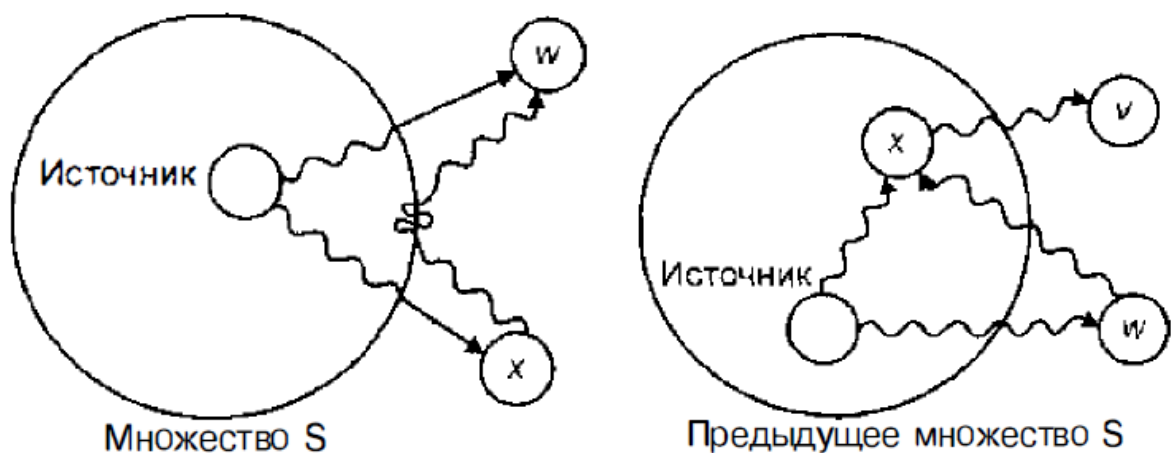
Предположим, существует более короткий путь P из s в u , который мы "пропустили". Этот путь должен где-то выйти за пределы S :

- $x \in S$ (последняя посещённая вершина на пути)
- $y \notin S$ (первая непосещённая вершина на пути)

Тогда:

1. $D[y] \leq D[x] + w(x, y)$ (потому что когда обрабатывали x , обновили y)
2. $D[u] \leq D[y]$ (потому что u выбрана как вершина с минимальным $d[]$ вне S)
3. Длина пути P : $L(P) \geq d[x] + w(x, y) + \dots \geq d[y] \geq d[u]$

Вывод: $L(P) \geq d[u] \rightarrow$ значит, более короткого пути нет, и $d[u]$ — окончательное!



6. Как работает алгоритм поиска подстроки Кнута-Морриса-Пратта (КМП).

Используем префикс функцию и при несовпадении сдвигаемся не на 1, а на длину префикса.

```
def prefix_func(s):
    pi = [0] * len(s)
    k = 0
    for i in range(1, len(s)):
        while k > 0 and s[k] != s[i]:
            k = pi[k-1]
        if s[k] == s[i]:
            k += 1
        pi[i] = k
    return pi
```

```
def KMP(text: str, pattern: str) -> list[int]:
    pref = pr(str(pattern))
    result = []
    i = 0
    j = 0
    while i < len(text):
        if text[i] == str(pattern)[j]:
            i += 1
            j += 1
            if j == 2:
                result.append(i)
                j = pref[j-1]
        else:
            if j > 0:
                j = pref[j-1]
            else:
                i += 1
    return result
```

7. Объясните принцип работы алгоритма Бойера-Мура для поиска подстрок.

Сканирование слева направо, сравнение справа налево. Совмещается начало текста (строки) и шаблона, проверка начинается с последнего символа шаблона. Если символы совпадают, производится сравнение предпоследнего

символа шаблона и т. д. Если все символы шаблона совпали с наложенными символами строки, значит, подстрока найдена, и выполняется поиск следующего вхождения подстроки.

Если же какой-то символ шаблона не совпадает с соответствующим символом строки, шаблон сдвигается на *несколько* символов вправо, и проверка снова начинается с последнего символа.

Эти «несколько», упомянутые в предыдущем абзаце, вычисляются по двум эвристикам.

1. **Правило плохого символа** (Bad Character Rule)
2. **Правило хорошего суффикса** (Good Suffix Rule)

Правило плохого символа

При несовпадении символа $c = \text{text}[i+j]$:

- Если c есть в образце: сдвигаем так, чтобы **правый экземпляр** c в образце совпал с c в тексте
- Если c нет в образце: сдвигаем за этот символ

Правило хорошего суффикса

При несовпадении s уже совпавшим суффиксом s :

- Ищем **самое правое вхождение** s в образце слева от текущей позиции
- Если нет полного вхождения, ищем совпадение суффикса s с префиксом образца

8. Докажите, что префикс-функция в алгоритме КМР работает за линейное время.

```
def prefix_func(s):
    pi = [0] * len(s)
    k = 0
    for i in range(1, len(s)):
        while k > 0 and s[k] != s[i]:
            k = pi[k-1]
        if s[k] == s[i]:
            k += 1
        pi[i] = k
    return pi
```


Префикс-функция для строки s — это массив pi , где каждый элемент $pi[i]$ показывает длину наибольшего *собственного* суффикса подстроки $s[0:i]$, который совпадает с её префиксом.

Шаги для каждого символа $s[i]$ (слева направо):

1. Пока есть несовпадение ($s[k] \neq s[i]$): "откатываем" k назад, используя ранее вычисленные значения pi . Это позволяет найти меньший префикс, который всё ещё является суффиксом текущей подстроки.
2. Если символы совпали ($s[k] = s[i]$), увеличиваем k на 1 (расширяем совпадающий префикс/суффикс).
3. $pi[i] = k$ (текущая длина совпадения).

Доказательство линейности ($O(n)$)

- Внешний цикл `for`: выполняется n раз.
- Внутренний цикл `while`: каждое выполнение уменьшает k минимум на 1.
- k увеличивается только во внешнем цикле (максимум на n за весь алгоритм). Значит, суммарно `while` выполнится не более n раз.

9. Сравните эффективность алгоритмов Рабина-Карпа и КМР для разных типов данных.

Алгоритм Рабина-Карпа использует скользящий хэш для ускорения сравнения, КМР использует префикс-функцию для ускорения передвижения по строке.

Алгоритм	Предобработка	Поиск	Память
РК	$O(m)$	$O(n \cdot m)$ (худший)	$O(1)$
КМР	$O(m)$	$O(n)$	$O(m)$

Где n — длина текста, m — длина шаблона.

Длина шаблона. При коротком шаблоне КМР может быть избыточен, РК быстрее сравнивает. При длинном шаблоне КМР эффективнее, РК сложнее хэшировать.

Частые совпадения в тексте. Префикс-функция КМР позволяет делать большие скачки после несовпадения 'В', минимизируя сравнения.

Шаблон с повторяющимися префиксами/суффиксами идеален для КМР.

Множественный поиск лучше для РК.

10. Поиск в глубину и в ширину. Примеры.

Оба алгоритма (DFS - Depth-First Search, BFS - Breadth-First Search) систематически посещают все вершины, достижимые из стартовой вершины, используя разные стратегии порядка обхода.

- BFS использует очередь (Queue, FIFO - First In First Out)
- DFS использует стек (Stack, LIFO - Last In First Out)

```
def bfs(graph, start):
    visited = set()      # Множество посещенных вершин
    queue = deque()      # Очередь для обработки вершин (FIFO)
    result = []          # Результат обхода

    queue.append(start)  # Добавляем стартовую вершину в очередь
    visited.add(start)   # Помечаем как посещенную

    while queue:
        vertex = queue.popleft() # Извлекаем вершину из начала очереди
        result.append(vertex)    # Добавляем в результат

        # Перебираем всех соседей текущей вершины
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)    # Помечаем соседа как посещенного
                queue.append(neighbor)   # Добавляем в конец очереди

    return result
```

```
# Пример графа в виде списка смежности
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}
```

Стр. 217 учебника.

11. Сравните устойчивые и неустойчивые алгоритмы сортировки. Приведите примеры и объясните, почему это свойство важно в реальных приложениях.

Устойчивые сохраняют относительный порядок элементов с одинаковыми ключами.

Неустойчивые могут изменять порядок равных элементов.

Примеры:

- Устойчивые: **MergeSort, InsertionSort, BubbleSort, Timsort.**
- Неустойчивые: **QuickSort, HeapSort, SelectionSort.**

Важность в реальных приложениях:

- **Многоуровневая сортировка:** Например, сначала сортируем по дате, затем по имени. Устойчивость гарантирует, что порядок имен внутри одной даты сохранится.
- **Визуализация:** Если элементы имеют одинаковое значение (например, высота столбцов), их исходный порядок может нести дополнительную информацию.

12. Разберите алгоритм Timsort (гибридный метод, используемый в Python). Как он сочетает сортировку вставками и слиянием?

1. **Разделение на прогоны (runs):**

- Ищет в массиве упорядоченные подпоследовательности (возрастающие или строго убывающие с последующим реверсом).

2. **Оптимизация прогонов:**

- Если прогон короче минимального размера (обычно 32–64 элемента), он дополняется следующими элементами и сортируется **вставками**.

3. **Слияние прогонов:**

- Прогоны объединяются методом **сортировки слиянием** с учетом их размеров. Используется стратегия *galloping mode*: если один прогон долго "выигрывает", алгоритм переходит к бинарному поиску для ускорения.

Почему эффективен?

- Использует существующую упорядоченность данных.

- Сортировка вставками эффективна на малых массивах (низкие накладные расходы).
- Слияние оптимизировано для минимизации сравнений и перемещений данных.

13. Алгоритмы сортировок QuickSort, HeapSort и InsertionSort.

Критерий	QuickSort	HeapSort	InsertionSort
Асимптотика	$O(n \log n)$ (средний), $O(n^2)$ (худ.)	$O(n \log n)$ (всегда)	$O(n^2)$
Память	$O(\log n)$ (стек вызовов)	$O(1)$	$O(1)$
Устойчивость	✗ Неустойчивый	✗ Неустойчивый	✓ Устойчивый
Оптимальные случаи	Частично отсортированные данные	Нет зависимости от данных	Почти отсортированные данные
Использование	Стандартная сортировка в C++, Rust	Там, где нужна гарантия $O(n \log n)$	Массивы ≤ 50 элементов

14. Как кеширование и локальность данных влияют на производительность сортировки слиянием (MergeSort) по сравнению с быстрой сортировкой (QuickSort)?

Понятие локальности

Локальность данных — свойство, при котором программа обращается к данным, находящимся рядом в памяти (пространственная локальность), или использует одни и те же данные многократно в короткий промежуток времени (временная локальность).

Кэширование — механизм хранения данных в быстрой памяти (кэше процессора) для ускорения доступа.

MergeSort

1. Объединение двух массивов требует обращения к разным частям памяти.
2. В рекурсивной реализации создаются новые временные массивы.
3. Плохая пространственная локальность из-за доступа к нерасположенным последовательно данным.
4. Большое количество копирований снижает эффективность кэширования.

QuickSort

- Работает *in-place*, т.е. сортирует без дополнительных массивов.
- Разбиение и рекурсивные вызовы проходят по соседним участкам памяти.
- Лучше использует кэш процессора благодаря высокой пространственной локальности.

Вывод: хотя MergeSort теоретически стабилен и всегда $O(n \log n)$, на практике QuickSort часто быстрее за счёт более эффективного использования кэша и памяти.

15. Почему сортировка выбором (SelectionSort) менее эффективна, чем сортировка вставками (InsertionSort), несмотря на одинаковую асимптотику $O(n^2)$?

1. **Количество операций записи:**
 - SelectionSort делает **$O(n)$ перестановок** (каждый элемент перемещается 1 раз).
 - InsertionSort делает **$O(n^2)$ присваиваний** (сдвигает элементы в цикле).
2. **Локальность данных:**
 - InsertionSort обрабатывает элементы последовательно → лучше использует кэш CPU.
 - SelectionSort скачет по массиву в поиске минимума → высокая вероятность **cache misses**.
3. **Адаптивность:**
 - InsertionSort эффективен на почти отсортированных данных ($O(n)$).
 - SelectionSort всегда выполняет полный перебор → $O(n^2)$ даже для отсортированного массива.

16. Как работает блочная сортировка (BucketSort) для равномерно распределённых данных? Приведите пример, когда она даёт $O(n)$.

Условие для $O(n)$: Данные должны быть **равномерно распределены** на интервале $[a, b]$.

Алгоритм:

1. Создать k пустых "ведер" (диапазонов).
2. Распределить элементы по ведрам: элемент x попадает в ведро с индексом $\text{floor}(k * (x - a) / (b - a))$.
3. Отсортировать каждое ведро **вставками** (или другим методом).
4. Объединить ведра в порядке возрастания индексов.

Пример за $O(n)$:

Пусть данные равномерно распределены на $[0, 1)$, и мы используем $k = n$ ведер.

- Распределение по ведрам: $O(n)$.
- В каждом ведре в среднем **1 элемент** $\rightarrow$ сортировка каждого ведра $O(1)$, общая $O(n)$.
- Объединение: $O(n)$.

Итого: $O(n)$.

Важно: Если данные распределены неравномерно (например, все в одном ведре), сложность деградирует до $O(n^2)$.

17. Алгоритмы хэширования SHA.

SHA-0 (1993): 160 битный хэш, уязвим.

SHA-1 (1995): взломан в 2017.

SHA-2 (2001): популярный SHA-256.

SHA-3 (Кескак, 2015): Победитель конкурса NIST.

1. Предобработка:

- Добавляем бит 1 к сообщению
- Дополняем нулями до длины $(\text{длина} + 64) \equiv 0 \pmod{512}$
- Добавляем 64-битную длину исходного сообщения

2. Разбиение: делим сообщение на 512-битные блоки

3. **Инициализация:** устанавливаем 8 начальных 32-битных значений (константы из дробных частей $\sqrt{p}$ простых чисел)

4. Каждый 512-битный блок обрабатывается в 64 раундах

Каждый раунд использует **1 слово данных (W[i]) + 1 константу (K[i])** для нелинейного преобразования 256-битного состояния. После 64 раундов состояние смешивается с исходным хешем. 512 бит блока "сжимаются" в 256-битное состояние следующего блока.

18. Алгоритмы хэширования CRC.

Применяется для защиты данных и обнаружении ошибок в потоке информации. Стал известен благодаря следующим качествам: легко и быстро обнаруживает ошибки, требует минимальные расходы, прост в эксплуатации.

Для определенной последовательности битов a_n составляется полином вида:

$$\sum_{n=0}^{N-1} a_n x^n$$

Значение $CRC(R(x))$ получается по следующей формуле:

$$R(x) = P(x) \cdot x^N \bmod G(x)$$

$P(x)$ – многочлен, представляющий из себя входные данные, $G(x)$ – порождающий многочлен, N – степень порождающего многочлена.

Популярные $G(x)$:

Название	Представления: нормальное / реверсированное / реверсированное от обратного
CRC-1	0x1 / 0x1 / 0x1
CRC-7	0x09 / 0x48 / 0x44
CRC-8-Dallas/Maxim	0x31 / 0x8C / 0x98
CRC-8	0xD5 / 0xAB / 0xEA
CRC-12	0x80F / 0xF01 / 0xC07
CRC-16-IBM	0x8005 / 0xA001 / 0xC002
CRC-16-CCITT	0x1021 / 0x8408 / 0x8810
CRC-16-T10-DIF	0x8BB7 / 0xEDD1 / 0xC5DB
CRC-16-DNP	0x3D65 / 0xA6BC / 0x9EB2
CRC-32-IEEE 802.3	0x04C11DB7 / 0xEDB88320 / 0x82608EDB
CRC-32C (Castagnoli)	0x1EDC6F41 / 0x82F63B78 / 0x8F6E37A0
CRC-32K (Koopman)	0x741B8CD7 / 0xEB31D82E / 0xBA0DC66B
CRC-32Q	0x814141AB / 0xD5828281 / 0xC0A0A0D5
CRC-64-ISO	0x0000000000000001B / 0xD800000000000000 / 0x800000000000000D